

Capitolo 8 - Stallo dei processi

Status Done

Modello di sistema

Per analizzare il problema dello stallo, è necessario definire un modello del sistema in termini di risorse.

- **Risorse:** il sistema è composto da un numero finito di risorse che devono essere distribuite tra i thread in competizione.
- **Tipi di risorsa:** le risorse sono suddivise in tipi (o classi), ciascuno dei quali è formato da un certo numero di **istanze identiche**. Esempi di tipi di risorsa includono cicli di CPU, file, dispositivi di I/O (come interfacce di rete o lettori DVD) e strumenti di sincronizzazione (es. lock mutex, semafori).
- **Ciclo di utilizzo:** un thread utilizza una risorsa seguendo una sequenza di tre passi:
 1. **Richiesta (Request):** il thread chiede un'istanza della risorsa. Se la richiesta non può essere soddisfatta immediatamente, il thread passa allo stato d'attesa (bloccato).
 2. **Uso (Use):** il thread utilizza la risorsa.
 3. **Rilascio (Release):** il thread rilascia la risorsa.

Situazioni di stallo in applicazioni multithread

Una situazione di stallo (deadlock) si verifica quando un insieme di thread attende indefinitamente un evento che può essere causato solo da un altro thread all'interno dello stesso insieme.

- **Causa:** lo stallo si verifica spesso quando i thread cercano di acquisire risorse detenute da altri thread che, a loro volta, sono in attesa di altre risorse.
- **Difficoltà di test:** identificare e sottoporre a test gli stalli è difficile, perché essi si verificano solo in determinate condizioni di scheduling specifiche.

Esempio: consideriamo due thread, `thread_one` e `thread_two`, e due lock mutex, `first_mutex` e `second_mutex`.

Tempo	<code>thread_one</code>	<code>thread_two</code>
T0	Acquisisce <code>first_mutex</code>	Acquisisce <code>second_mutex</code>
T1	Richiede <code>second_mutex</code>	Richiede <code>first_mutex</code>

⇒ Entrambi i thread sono bloccati: `thread_one` attende `second_mutex` (detenuto da `thread_two`), e `thread_two` attende `first_mutex` (detenuto da `thread_one`). La situazione è di stallo.

Situazioni di stallo attivo (livelock)

Lo stallo attivo è una forma di mancanza di liveness strettamente correlata allo stallo, ma con una differenza cruciale:

- **Definizione:** in una situazione di livelock, i thread non sono bloccati in senso stretto, ma sono attivi e impegnati a cambiare il loro stato. Tuttavia, non riescono comunque a progredire nell'esecuzione.
- **Causa:** il livelock si verifica quando l'algoritmo di prevenzione o recupero dallo stallo richiede ai thread di rilasciare temporaneamente le risorse e ritentare l'acquisizione, ma lo fanno in sincronia in un modo che garantisce che nessuno abbia successo.
- **Differenza dallo stallo:** in uno stallo, i thread sono in attesa statica. In un livelock, i thread eseguono attivamente le istruzioni, ma il loro sforzo è inutile e non porta a nessun progresso.

Esempio di Livelock (Pthreads `trylock`)

Se nell'esempio di stallo precedente i thread avessero usato un approccio che prevedeva di rilasciare il primo lock se non si riusciva ad acquisire il secondo lock, avrebbero potuto entrare in livelock:

1. `thread_one` prende `first_mutex`, `thread_two` prende `second_mutex`.
2. `thread_one` fallisce nell'acquisire `second_mutex`, rilascia `first_mutex` e ripete.
3. `thread_two` fallisce nell'acquisire `first_mutex`, rilascia `second_mutex` e ripete.

Se questi eventi si ripetono in perfetto sincronismo, nessuno dei due thread avanzerà, pur eseguendo attivamente codice (cioè, sono in livelock).

Caratterizzazione delle situazioni di stallo

Lo stallo (deadlock) è possibile solo se nel sistema si verificano simultaneamente quattro condizioni specifiche. Se anche una di queste condizioni non è soddisfatta, lo stallo è impossibile.

Condizioni necessarie

Le quattro condizioni necessarie sono:

1. **Mutua esclusione:**

- Almeno una risorsa deve essere non condivisibile (non-sharable), ovvero può essere utilizzata da un solo thread alla volta.
- Se un altro thread la richiede, deve attendere il rilascio della risorsa.
- Es. un lock mutex o una singola stampante.

2. **Possesso e attesa (Hold and Wait):**

- Un thread deve essere in possesso di almeno una risorsa.
- Allo stesso tempo, deve essere in attesa di acquisire altre risorse che sono già in possesso di altri thread.

3. **Assenza di prelazione (no preemption):**

- Una risorsa non può essere tolta con la forza (prelazionata) al thread che la possiede.
- La risorsa può essere rilasciata dal thread che la possiede solo volontariamente, e solo dopo che il thread ha completato il suo compito che richiede tale risorsa.

4. **Attesa circolare (Circular Wait):**

- Deve esistere una catena circolare di due o più thread.
- In questa catena, ogni thread attende una risorsa detenuta dal thread successivo della catena.
- Formalmente:

Esiste un insieme di thread $\{T_0, T_1, \dots, T_n\}$ tale che T_0 attende una risorsa detenuta da T_1 , T_1 attende una risorsa detenuta da T_2 , e così via, fino a che T_{n-1} attende una risorsa detenuta da T_n , e T_n attende una risorsa detenuta da T_0 .

Grafo di assegnazione delle risorse (Resource-Allocation Graph)

Il grafo di assegnazione delle risorse è uno strumento grafico formale utilizzato per modellare il modo in cui le risorse sono assegnate ai thread.

Componenti del grafo

Il grafo è composto da un insieme di vertici V e un insieme di archi E .

1. **Vertici:** sono di due tipi:

- **Thread (T):** cerchi (o ovali), che rappresentano i thread nel sistema.
- **Tipi di risorsa (R):** quadrati, che rappresentano i tipi di risorsa nel sistema. All'interno del quadrato possono essere mostrati dei punti per indicare il numero di istanze disponibili per quel tipo di risorsa.

2. **Archivi:** sono di due tipi:

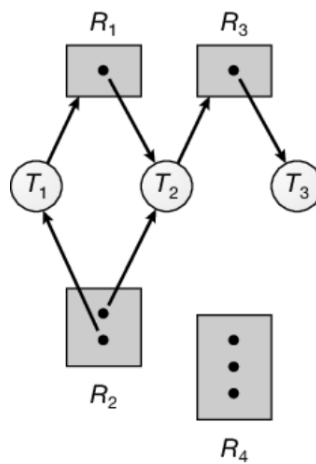
- **Arco di richiesta (Request Edge):** un arco diretto da un thread T_i a un tipo di risorsa R_j ($T_i \rightarrow R_j$). Indica che il thread T_i sta richiedendo un'istanza della risorsa R_j ed è in attesa.
- **Arco di assegnazione (Assignment Edge):** un arco diretto da un tipo di risorsa R_j a un thread T_i ($R_j \rightarrow T_i$). Indica che un'istanza della risorsa R_j è stata assegnata al thread T_i .

Rilevamento dello stallo tramite il grafo

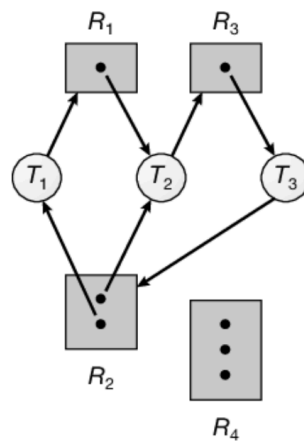
La presenza di cicli nel grafo è fondamentale per determinare se si è verificato uno stallo:

- Se il grafo non contiene cicli, allora non c'è stallo nel sistema.
- Se il grafo contiene un ciclo, può esserci stallo.
 - Se ogni tipo di risorsa nel ciclo ha esattamente una sola istanza, allora l'esistenza di un ciclo implica la presenza di stallo.
 - Se più istanze sono disponibili per almeno un tipo di risorsa nel ciclo, l'esistenza di un ciclo è solo una condizione necessaria ma non sufficiente per lo stallo. Lo stallo potrebbe non verificarsi se un thread non incluso nel ciclo rilascia una risorsa che può interrompere l'attesa circolare.

Esempio:



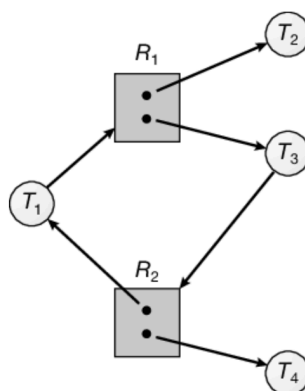
Si supponga che il thread T_3 richieda un'istanza del tipo di risorsa R_2 . Poiché non è disponibile alcuna istanza di risorsa, si aggiunge un arco di richiesta.



I thread T_1, T_2 e T_3 sono in stallo:

- T_2 attenda R_3 , posseduta da T_3 ;
- T_3 attende che T_1 o T_2 rilasci R_2 ;
- T_1 attende che T_2 rilasci R_1 .

Esempio:



Anche in questo caso è presente un ciclo: $T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$.

In questo caso, però, non è presente stallo: il thread T_4 può rilasciare la propria istanza del tipo di risorsa R_2 , che si può assegnare al

thread T_3 , rompendo il ciclo.

Metodi per la gestione degli stalli

Quando si progetta un sistema, si può scegliere quattro strategie per affrontare il problema del deadlock:

1. Prevenzione degli stalli (Deadlock Prevention)

Questo approccio garantisce che lo stallo non possa mai verificarsi a priori.

- **Strategia:** prevenire lo stallo significa negare (o assicurarsi che non sia mai soddisfatta) almeno una delle quattro condizioni necessarie per lo stallo (mutua esclusione, possesso e attesa, assenza di prelazione, attesa circolare).
- **Costo:** spesso, la prevenzione può portare a un basso utilizzo delle risorse e a una riduzione del throughput del sistema, poiché impone vincoli molto rigidi sull'uso delle risorse. Ad esempio, una strategia di prevenzione potrebbe costringere un thread a richiedere tutte le risorse necessarie contemporaneamente, anche se ne avrà bisogno solo in seguito.

2. Evitamento degli stalli (Deadlock Avoidance)

Questo approccio consente che le quattro condizioni necessarie per lo stallo possano esistere, ma impedisce al sistema di entrare in uno stato non sicuro.

- **Strategia:** il SO controlla ogni richiesta di risorsa in modo dinamico. Solo se l'assegnazione della risorsa può mantenere il sistema in uno stato sicuro (safe state), la richiesta viene soddisfatta. Uno stato sicuro è quello in cui esiste almeno una sequenza di esecuzione dei thread che permette a tutti di completare le loro operazioni.
- **Informazioni aggiuntive:** l'evitamento richiede che il sistema conosca in anticipo il fabbisogno massimo di risorse di ogni thread.
- **Costo:** l'algoritmo di evitamento (come l'algoritmo del banchiere) comporta un **overhead computazionale** maggiore rispetto alla prevenzione, poiché deve analizzare lo stato del sistema ad ogni richiesta.

3. Rilevamento e recupero degli stalli (Deadlock Detection and Recovery)

Questo approccio permette al sistema di entrare in uno stato di stallo, ma fornisce gli strumenti per identificarlo e risolverlo.

- **Strategia:** il sistema esegue periodicamente un algoritmo di rilevamento per scoprire se si è formato uno stallo.
- **Recupero:** se uno stallo viene rilevato, il sistema deve adottare un meccanismo di recupero per interrompere il circolo vizioso. I metodi di recupero includono:
 - **Terminazione del thread:** terminare forzatamente uno o più thread coinvolti nello stallo.
 - **Prelazione delle risorse:** togliere con la forza una risorsa a un thread (rollout) e assegnarla a un altro.
- **Costo:** questo approccio ha il costo della costante esecuzione dell'algoritmo di rilevamento e del rollback o della terminazione dei thread durante il recupero.

4. Ignoranza del problema (Ignorance)

Questo è il metodo più semplice, utilizzato dalla maggior parte dei SO generici, inclusi UNIX, Linux, Windows.

- **Strategia:** si assume che gli stalli non si verificheranno mai. Il SO non fornisce alcuna strategia interna per la prevenzione, l'evitamento o il rilevamento degli stalli.
- **Motivazione:** si ritiene che gli stalli siano rari o che il costo e l'overhead di prevenzione/rilevamento superino i benefici. L'onere di evitare gli stalli è lasciato agli sviluppatori di applicazioni.
- **Risoluzione:** se uno stallo si verifica, richiede un riavvio manuale del sistema o la terminazione del thread da parte dell'utente.

Prevenzione degli stalli

La prevenzione degli stalli mira a negare in modo proattivo almeno una delle quattro condizioni necessarie, rendendo lo stallo strutturalmente impossibile.

Condizione Necessaria	Strategia di Prevenzione	Vantaggi e Svantaggi Chiave
Mutua Esclusione	Negazione della non-condivisibilità	Strategia: Rendere le risorse condivisibili, se possibile (es. file in sola lettura). Limiti: Impossibile per risorse intrinsecamente non condivisibili (es. mutex, stampanti).
Possesso e Attesa	Richiesta 'All-or-Nothing' o Rilascio Forzato	Protocolli: 1. Richiedere tutte le risorse prima di iniziare. 2. Rilasciare tutte le risorse detenute prima di richiederne di nuove. Costi: Basso utilizzo delle risorse (detenute inutilmente) e rischio di starvation .

Assenza di Prelazione	Prelazione Implicita	Protocolli: 1. Rilascio: Se la richiesta fallisce, il thread rilascia le risorse detenute e riprova. 2. Assegnazione: Prelazionare risorse da un thread in attesa per assegnarle a un richiedente. Costi: Difficile da implementare per risorse il cui stato è difficile da salvare e ripristinare (es. lock).
Attesa Circolare	Ordinamento Gerarchico delle Risorse	Strategia: Imporre un ordine numerico rigido a tutti i tipi di risorsa ($R_1, R_2, \dots$). Protocollo: I thread devono richiedere le risorse solo in ordine crescente ($F(R_j) > F(R_i)$). Vantaggio: Elimina la possibilità di cicli. Costo: Inefficienza, poiché i thread potrebbero essere costretti a richiedere risorse prima del momento ottimale.

Evitamento degli stalli

È una strategia che consente le condizioni necessarie per lo stallo, ma impedisce al sistema di raggiungerlo.

- **Requisito:** il SO deve conoscere in anticipo il fabbisogno massimo di risorse di ciascun thread.
- **Decisione dinamica:** ad ogni richiesta di risorsa, il sistema decide se l'allocazione può portare a uno stallo.

Stato sicuro

- **Stato sicuro:** lo stato attuale del sistema è sicuro se esiste una sequenza sicura di esecuzione dei thread, che garantisce che tutti i thread possano completare il loro lavoro (anche nel caso peggiore) utilizzando le risorse disponibili e quelle rilasciate dai thread che hanno finito.
- **Stato non sicuro:** uno stato che non garantisce il completamento di tutti i thread e può condurre allo stallo. Il sistema non deve mai passare a uno stato non sicuro.

Algoritmi di evitamento

L'algoritmo utilizzato dipende dal numero di istanze per tipo di risorsa:

- **Istanza singola:** si utilizza l'analisi del **grafo di assegnazione delle risorse**, introducendo "archi di richiesta potenziale" per tracciare le future esigenze. L'allocazione è negata se crea un ciclo.
- **Istanze multiple:** si utilizza l'**algoritmo del banchiere**.

Algoritmo del banchiere

Applicato a tipi di risorsa con più istanze.

Deve garantire che, dopo ogni allocazione di risorse, il sistema rimanga in uno stato sicuro, in cui esiste almeno una sequenza di esecuzione che permette a tutti i thread di terminare il loro lavoro.

Strutture dati

L'algoritmo si basa sul monitoraggio continuo di quattro strutture dati fondamentali:

1. `available[j]`: (vettore), è il numero di istanze del tipo di risorsa R_j attualmente disponibili.
2. `max[i, j]`: (matrice $n \times m$), è il numero massimo di istanze di R_j che il thread T_i potrà mai richiedere.
3. `allocation[i, j]`: (matrice $n \times m$), è il numero massimo di istanze di R_j attualmente assegnate al thread T_i .
4. `need[i, j]`: (matrice $n \times m$), è il numero di istanze di R_j che il thread T_i ha ancora bisogno di acquisire per completare il suo compito. Si calcola come:

$$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$$

Logica dell'algoritmo (due fasi)

Quando un thread T_i richiede un vettore di risorse Request_i , l'algoritmo esegue due controlli:

- **Algoritmo di richiesta di risorsa**

Questo è il protocollo per decidere se procedere con l'allocazione:

1. **Verifica 1 (Validità):** se Request_i è maggiore di Need_i , la richiesta è invalida (il thread non può chiedere più di quanto ha dichiarato come massimo). → Errore.
2. **Verifica 2 (Disponibilità immediata):** se Request_i è maggiore di Available , il thread attende (le risorse non sono sufficienti).

3. **Test ipotetico:** se le prime due verifiche passano, il sistema simula l'allocazione (esegue il test provvisorio):

$$\begin{aligned}\text{Available} &= \text{Available} - \text{Request}_i \\ \text{Allocation}_i &= \text{Allocation}_i + \text{Request}_i \\ \text{Need}_i &= \text{Need}_i - \text{Request}_i\end{aligned}$$

4. **Verifica sicurezza:** esegue l'algoritmo di sicurezza sul nuovo stato simulato.

5. **Conclusione:**

- Se lo stato è sicuro, l'allocazione simulata è resa permanente.
- Se lo stato è non sicuro, l'allocazione viene annullata (rollback) e il thread attende.

• **Algoritmo di sicurezza (ricerca di una sequenza sicura)**

Questo algoritmo è eseguito sul nuovo stato simulato per verificare se è possibile trovare una sequenza sicura $\langle T_{i_1}, T_{i_2}, \dots, T_{i_n} \rangle$:

1. **Inizializzazione:**

- Vettore di lavoro $\text{Work} = \text{Available}$.
- Vettore booleano Finish (dimensione n), inizializzato a false .

2. **Ricerca:** trova un indice i tale che:

- $\text{Finish}[i]$ è false (il thread non ha ancora completato).
- Need_i è minore o uguale a Work (le risorse attualmente disponibili sono sufficienti per T_i per completare).

3. **Completamento simulato:** se un tale thread T_i viene trovato:

- Si assume che T_i completerà, rilasciando tutte le sue risorse.
- $\text{Work} = \text{Work} + \text{Allocation}_i$ (le risorse di T_i vengono aggiunte a quelle disponibili).
- $\text{Finish}[i] = \text{true}$
- Si torna al passo 2 (cercare il thread successivo).

4. **Conclusione:** se $\text{Finish}[i]$ è true per tutti gli i (cioè, tutti i thread possono completare), lo stato è sicuro. Altrimenti, è non sicuro.

Esempio:

- Stato iniziale del sistema (T_0): il sistema è composto da 5 processi e 3 tipi di risorsa: A (10 istanze), B (5 istanze), e C (7 istanze).

Processo	Allocation (A B C)	Max (A B C)	Available (A B C)
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

- Calcolo della matrice need: la matrice need è calcolata come Max - Allocation:

Processo	Need (A B C) = Max - Allocation
P_0	$753 - 010 = 7\ 4\ 3$
P_1	$322 - 200 = 1\ 2\ 2$
P_2	$902 - 302 = 6\ 0\ 0$
P_3	$222 - 211 = 0\ 1\ 1$
P_4	$433 - 002 = 4\ 3\ 1$

- **Verifica di Sicurezza (Algoritmo di Sicurezza):** per determinare se lo stato T_0 è sicuro, cerchiamo una **sequenza sicura** di esecuzione (dove un thread può completare se $\text{Need} \leq \text{Available}$).

1. **Stato Iniziale:** $\text{Work} = (3, 3, 2)$.

2. **Trova un Thread Eseguitibile:** solo P_1 (Need: 1 2 2) e P_3 (Need: 0 1 1) hanno un fabbisogno $\leq \text{Work}$ (3 3 2). Scegliamo P_1 .

3. P_1 Esegue:

- $Work = Work + Allocation_{P_1}$
- $Work = (3, 3, 2) + (2, 0, 0) = (5, 3, 2)$.

4. Trova un Thread Eseguitibile (Work: 5 3 2):

- P_3 (Need: 0 1 1) è eseguibile.

5. P_3 Esegue:

- $Work = (5, 3, 2) + (2, 1, 1) = (7, 4, 3)$.

6. ...

Conclusione: La sequenza $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ (o qualsiasi sua permutazione valida) soddisfa il requisito di sicurezza. Il sistema è in uno stato sicuro.

- Simuliamo la richiesta di P_1 per $Request_{P_1} = (1, 0, 2)$.
 - Controlli iniziali
 1. **Validità:** $Request \leq Need_{P_1}$? $(1, 0, 2) \leq (1, 2, 2)$
 2. **Disponibilità:** $Request \leq Available$? $(1, 0, 2) \leq (3, 3, 2)$
 - Stato simulato: simuliamo l'allocazione, portando il sistema in uno stato ipotetico T_1 :

Matrice	Vecchio Stato	Richiesta	Nuovo Stato
Available	3 3 2	- (1 0 2)	2 3 0
Allocation _{P₁}	2 0 0	+ (1 0 2)	3 0 2
Need _{P₁}	1 2 2	- (1 0 2)	0 2 0

- Verifica sicurezza sullo stato simulato: eseguiamo l'Algoritmo di Sicurezza con $Work = (2, 3, 0)$:
 1. **Trova un Thread Eseguitibile:** Cerchiamo T_i tale che $Need_i \leq (2, 3, 0)$.
 2. P_1 (Need: 0 2 0) è eseguibile (è soddisfatto, e può terminare).
 3. P_3 (Need: 0 1 1) è eseguibile (1 non è ≤ 0 in C).
 4. P_4 (Need: 4 3 1) è eseguibile.

L'unico thread che può completare subito è P_1 . Se P_1 termina, rilascia $Allocation_{P_1} = (3, 0, 2)$.

- $Work = (2, 3, 0) + (3, 0, 2) = (5, 3, 2)$.

Adesso cerchiamo un thread eseguibile con $Work = (5, 3, 2)$:

- P_3 (Need: 0 1 1) è eseguibile. Rilascia (2, 1, 1). $Work = (7, 4, 3)$.
- P_4 (Need: 4 3 1) è eseguibile. Rilascia (0, 0, 2). $Work = (7, 4, 5)$.
- P_0 (Need: 7 4 3) è eseguibile. Rilascia (0, 1, 0). $Work = (7, 5, 5)$.
- P_2 (Need: 6 0 0) è eseguibile. Rilascia (3, 0, 2). $Work = (10, 5, 7)$.

Conclusione: Una sequenza sicura $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ è stata trovata. Lo stato simulato è sicuro.

Risposta Finale: La richiesta di P_1 per $\langle 1, 0, 2 \rangle$ viene accolta immediatamente.

Rilevamento degli stalli (Deadlock Detection)

Se un sistema non utilizza strategie di prevenzione o evitamento, deve fornire un meccanismo per rilevare se uno stallo è in corso e un meccanismo per recuperare da esso.

- **Strategia:** l'algoritmo di rilevamento esamina lo stato del sistema (allocazione, richieste e disponibilità delle risorse) per determinare se esiste una condizione di attesa circolare.
- **Frequenza:** il rilevamento può essere eseguito periodicamente, ad esempio ogni volta che una richiesta di risorsa fallisce e causa il blocco di un thread, o a intervalli di tempo regolari. La frequenza è un compromesso tra l'overhead computazionale e la tempestività con cui lo stallo viene identificato.

Algoritmo di rilevamento a istanza singola

Quando ogni tipo di risorsa ha una sola istanza, l'algoritmo di rilevamento è particolarmente semplice: si basa sulla ricerca di cicli nel grafo di assegnazione delle risorse.

- **Metodo:** l'algoritmo cerca cicli nel grafo. Se esiste un ciclo, allora si è verificato uno stallo.
- **Costo:** l'algoritmo può essere implementato in modo efficiente (ad esempio, tramite algoritmi standard di ricerca in profondità o ampiezza), con un costo computazionale di $O(n^2)$, dove n è il numero di thread.

Algoritmo di rilevamento a istanza multipla

Quando i tipi di risorsa possono avere più istanze (come nell'algoritmo del banchiere), la ricerca di cicli nel grafo non è più sufficiente. L'algoritmo di rilevamento deve essere più sofisticato e utilizza le stesse strutture dati del banchiere.

- **Strutture dati:** vettori e matrici che rappresentano lo stato del sistema:
 - **available**: risorse disponibili.
 - **allocation**: risorse attualmente detenute da ciascun thread.
 - **request**: risorse che ciascun thread sta attualmente richiedendo (è in attesa).
- **Funzionamento:** l'algoritmo cerca di trovare una sequenza in cui tutti i thread possono completare il loro compito (simile all'algoritmo di sicurezza del banchiere), utilizzando le risorse disponibili più quelle che un thread rilascia.
- **Rilevamento dello stallo:** se al termine dell'algoritmo non è possibile contrassegnare tutti i thread come "finiti", i thread rimanenti (quelli contrassegnati come false) sono coinvolti in una situazione di stallo.

▼ Esempio di esecuzione

Stato iniziale del sistema (T_0)

Il sistema è composto da cinque processi e tre tipi di risorse: A (7 istanze), B (2 istanze) e C (6 istanze).

Processo	Allocation (A B C)	Request (A B C)	Available (A B C)
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

L'algoritmo cerca una sequenza di esecuzione che permetta a tutti i processi di completare, assumendo che i processi completati rilascino le loro risorse.

1. **Inizializzazione:** $Work = Available = (0, 0, 0)$. $Finish[i] = false$ per tutti i processi.
2. **Passo 1 (Cerca P_i tale che $Request_i \leq Work$):** nessun processo può eseguire immediatamente, poiché $Work = (0, 0, 0)$ e tutti i processi tranne P_0 hanno richieste attive. Poiché P_0 ha $Request_{P_0} = (0, 0, 0)$, P_0 può completare subito (è l'unico thread che non sta aspettando risorse).
3. **P_0 esegue:**
 - $Work = Work + Allocation_{P_0}$
 - $Work = (0, 0, 0) + (0, 1, 0) = (0, 1, 0)$
4. **Resto dell'esecuzione:**
 - P_2 esegue: $Request_{P_2} = (0, 0, 0) \leq Work = (0, 1, 0)$ (Vero).
Rilascia (3, 0, 3).
 - P_3 esegue: $Request_{P_3} = (1, 0, 0) \leq Work = (3, 0, 3)$ (Vero).
Rilascia (5, 1, 4).
 - P_1 esegue: $Request_{P_1} = (2, 0, 2) \leq Work = (5, 1, 4)$ (Vero).
Rilascia (7, 1, 4).
 - P_4 esegue: $Request_{P_4} = (0, 0, 2) \leq Work = (7, 1, 4)$ (Vero).
Rilascia (7, 1, 6).

Conclusione T_0 : poiché la sequenza $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ esiste, lo stato al tempo T_0 è privo di stallo.

Richiesta aggiuntiva e rilevamento dello stallo

Il processo P_2 richiede un'istanza aggiuntiva del tipo di risorsa C. $Request_{P_2}$ diventa (0, 0, 1).

1. **Esecuzione dell'algoritmo di rilevamento (nuovo stato)**
Available: è ancora (0, 0, 0).

a. **Trova un thread eseguibile:** l'unico processo che può completare è P_0 che dopo l'esecuzione rilascia $\text{Allocation}_{P_0} = (0, 1, 0)$.

b. **Trova il thread successivo:** cerchiamo P_i tale che $\text{Request}_i \leq (0, 1, 0)$.

i. $\text{Request}_{P_3} = (1, 0, 0) \rightarrow \text{Falso}$.

ii. $\text{Request}_{P_1} = (2, 0, 2) \rightarrow \text{Falso}$.

iii. $\text{Request}_{P_2} = (0, 0, 1) \rightarrow \text{Falso}$.

iv. $\text{Request}_{P_4} = (0, 0, 2) \rightarrow \text{Falso}$.

2. Conclusione $\rightarrow$ Stallo:

- Stato del sistema: solo P_0 può completare.
- Risorse reclamabili: solo le risorse detenute da P_0 $(0, 1, 0)$ possono essere recuperate.
- Risorse insufficienti: con $\text{Work} = (0, 1, 0)$, non ci sono risorse sufficienti per soddisfare le richieste di P_1, P_2, P_3, P_4 .
- Risultato: esiste uno stallo costituito da $\{P_1, P_2, P_3, P_4\}$. Questi quattro processi si attendono reciprocamente in un ciclo, e il sistema non può garantire il loro completamento.

Recupero degli stalli (Recovery from Deadlock)

Una volta che uno stallo è stato rilevato, il sistema deve interrompere il circolo vizioso per riportare il sistema alla normalità. Esistono due metodi principali di recupero:

1. Terminazione del thread:

- **Terminazione di tutti i thread coinvolti:** l'approccio più semplice è terminare tutti i thread che sono coinvolti nello stallo. Questo è efficace, ma ha il costo di ricominciare il lavoro da zero.
- **Terminazione selettiva:** terminare un thread alla volta finché il ciclo di stallo non viene interrotto. Ad ogni terminazione, si esegue nuovamente l'algoritmo di rilevamento. La scelta del thread da terminare si basa su criteri che minimizzano il costo (es. thread con il tempo di esecuzione minore, thread che ha già consumato meno risorse).

2. Prelazione delle risorse (Resource Preemption):

- **Selezione della vittima:** quale risorsa prelevare da quale thread, la scelta è basata sul minimizzare i costi di ripristino.
- **Rollback:** il thread a cui è stata sottratta la risorsa deve essere ripristinato a uno stato precedente sicuro. La forma più semplice è un rollback totale (riavvio del thread).
- **Starvation (inedia):** bisogna assicurarsi che un thread non venga scelto ripetutamente come vittima per la prelazione, impedendogli di completare il suo lavoro.

Approccio alternativo

La maggior parte dei SO generici, inclusi UNIX, Linux, Windows, adotta un approccio semplice ma pragmatico per la gestione degli stalli: l'ignoranza del problema.

- **Strategia:** il SO assume che gli stalli non si verifichino mai nella pratica. Non implementa algoritmi interni di prevenzione, evitamento o rilevamento.
- **Motivazione:** si ritiene che il costo e l'overhead di esecuzione continua degli algoritmi di evitamento o rilevamento (come l'algoritmo del banchiere) siano superiori alla probabilità e al costo del verificarsi di uno stallo.
- **Conseguenza:** se si verifica uno stallo, il sistema entra in uno stato di non-liveness e richiede un riavvio manuale del sistema (o la terminazione forzata dei processi coinvolti) per risolvere il problema.

Questo approccio è considerato ragionevole perché gli stalli, sebbene teoricamente possibili, sono eventi rari nei SO moderni ben progettati.

Stalli a Livello Applicativo

Nonostante il kernel ignori gli stalli, essi possono e si verificano a livello di applicazione utente, soprattutto nelle applicazioni multi-thread che usano lock mutex e semafori.

- **Responsabilità:** l'onere della prevenzione e dell'evitamento ricade interamente sullo sviluppatore dell'applicazione.
- **Gestione in pratica:** gli sviluppatori devono garantire che le loro applicazioni non creino condizioni di stallo. Il metodo più comune per prevenire gli stalli a livello applicativo è imporre un ordinamento gerarchico (condizione di attesa circolare) per l'acquisizione dei lock, assicurando che i lock vengano sempre acquisiti nello stesso ordine logico.

Stalli in SO specifici

- **UNIX/Linux:** non forniscono strumenti per l'evitamento o il rilevamento degli stalli.
- **Windows:** non fornisce algoritmi per la prevenzione o il rilevamento generale degli stalli del SO. Tuttavia, fornisce strumenti e librerie (come le API di sincronizzazione C++) che consentono agli sviluppatori di applicazioni di implementare i propri meccanismi di prevenzione o rilevamento. Ad esempio, offre funzioni di timeout che permettono ai thread di provare ad acquisire un lock solo per un certo periodo, evitando attese infinite (pur non risolvendo la causa dello stallo).